A LoRe directly for language generation

A language model is a policy network that assigns the likelihood of a response $y \in \mathcal{Y}$, given prompt $x \in \mathcal{X}$ denoted by $\pi_\theta : \mathcal{X} \mapsto \Delta^{|\mathcal{Y}|-1}$. Given a reward function $r_\phi : \mathcal{X} \times \mathcal{Y} \mapsto \mathbb{R}$ and a reference policy π_{ref} (typically obtained by supervised finetuning a pre-trained model), the objective is to learn a policy π_θ that maximizes the KL-regularized reward maximization problem:

$$\theta^* = \arg\max_{\theta \in \Theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [r_\phi(x, y) - \beta \mathbb{D}_{\text{KL}}(\pi_\theta(y|x) || \pi_{\text{ref}}(y|x)]. \tag{16}$$

Rafailov et al. (2024) shows that the optimal policy for the KL-regularized reward maximization problem satisfies

$$\pi_{\theta^*}(y|x) = \frac{\pi_{\text{ref}}(y|x)}{Z_\phi(x)} \exp\left(\frac{r_\phi(x, y)}{\beta}\right) \implies r_\phi(x, y) = \beta \log \frac{\pi_{\theta^*}(y|x)}{\pi_{\text{ref}}(y|x)} + \beta Z_\phi(x), \tag{17}$$

where $Z_\phi(x)$ is a normalization factor. Since there is a direct correspondence between the reward function r_ϕ and the optimal policy that maximizes the KL-regularized reward, Rafailov et al. (2024) proposes Direct Preference Optimization (DPO), which directly learns the policy π_{θ^*} without first learning reward model r_ϕ on offline preference data. Then, by using this closed-form expression for $r_\phi(x, y)$ and applying minimizing negative log-likelihood to the Bradley-Terry model, we can obtain

$$\begin{aligned} &\min_{\phi \in \Phi} \sum_{(x, y_c, y_r) \in \mathcal{D}} \ell(r_\phi(x, y_c) - r_\phi(x, y_r)) \\ &= \min_{\theta \in \Theta} \sum_{(x, y_c, y_r) \in \mathcal{D}} \ell\left(\beta \log \frac{\pi_\theta(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_\theta(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right) \qquad \text{(by Eq. (17).)} \end{aligned}$$

Learning a Policy Basis: Instead of first learning a reward basis function $\mathbf{R}_\phi : \mathcal{X} \times \mathcal{Y} \mapsto \mathbb{R}^B$, it is equivalent to learning (fine-tuning) B basis policies $\{\pi_{\theta_i} : \mathcal{X} \mapsto \Delta^{|\mathcal{Y}|-1}\}_{i \in [B]}$, which produces B likelihoods, each corresponding to the optimal policy for each of the latent reward dimensions.

Conditioned on a prompt x, and a pair of responses $(y, \tilde{y})$, the probability of a user with preference weights $\mathbf{w} \in \Delta^{B-1}$ of choosing y over $\tilde{y}$ under the Bradley-Terry model can be written as:

$$\begin{aligned} \mathbb{P}[y \succ \tilde{y}|x] &= \frac{1}{1 + \exp\left(-\mathbf{w}^\top \left(\mathbf{R}_\phi(x, y) - \mathbf{R}_\phi(x, \tilde{y})\right)\right)} \\ &= \frac{1}{1 + \exp\left(-\beta \sum_{j \in [B]} w^{(j)} \left(\log \frac{\pi_{\theta_j}(y|x)}{\pi_{\text{ref}}(y|x)} - \log \frac{\pi_{\theta_j}(\tilde{y}|x)}{\pi_{\text{ref}}(\tilde{y}|x)}\right)\right)}, \qquad \text{(by Eq. (17).)} \end{aligned}$$

where $w^{(j)}$ is the jth entry of the vector $\mathbf{w}$. Analogous to Eq. (10), given a dataset of offline preferences, the parameters θ and $\{\mathbf{w}_i\}_{i \in \mathcal{U}_{\text{seen}}}$ can be learnt by plugging in the negative log-likelihood loss as

$$\min_{\{\theta_j : \Theta\}_{j \in [B]}, \{\mathbf{w}_i \in \Delta^{B-1}\}_{i \in \mathcal{U}_{\text{seen}}}} \sum_{i \in \mathcal{U}_{\text{seen}}} \frac{1}{|\mathcal{D}_i|} \sum_{(x, y_c, y_r) \in \mathcal{D}_i} \ell\left(\sum_{j \in [B]} w_i^{(j)} \left(\beta \log \frac{\pi_{\theta_j}(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_{\theta_j}(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right)\right). \tag{18}$$

For a new user, θ is frozen, and only the user weight $\mathbf{w}_{\text{new}}$ is estimated using few-shot examples:

$$\mathbf{w}_{\text{new}} = \operatorname*{argmin}_{\mathbf{w} \in \Delta^{B-1}} \sum_{(x, y_c, y_r) \in \mathcal{D}_i} \ell\left(\sum_{j \in [B]} w^{(j)} \left(\beta \log \frac{\pi_{\theta_j}(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_{\theta_j}(y_r|x)}{\pi_{\text{ref}}(y_r|x)}\right)\right). \tag{19}$$

B Additional Details on Experiments

Evaluation Metrics For any user $i \in \mathcal{U}_{\text{seen}} \cup \mathcal{U}_{\text{unseen}}$ (seen or unseen) we evaluate the performance of the learned reward basis $\mathbf{R}_\phi$ and preferences $\mathbf{w}_i$, on their unseen pairs of responses $\tilde{\mathcal{D}}_i$ (ie. data not present in $\mathcal{D}_{\text{train}}$ or

$\mathcal{D}_{\text{fewshot}}$) as the fraction of responses the learnt reward model classified correctly, defined formally as:

$$\frac{1}{|\tilde{\mathcal{D}}_i|} \sum_{(x,y_c,y_r)\in\tilde{\mathcal{D}}_i} \mathbb{I}[\mathbf{w}_i^\top(\mathbf{R}_\phi(x,y_c) - \mathbf{R}_\phi(x,y_r)) > 0]. \tag{20}$$

To test how well the learnt parameters generalize, we consider the following:

1. **Generalizing to Unseen Pairs of Responses for Seen Users (Seen Accuracy):** This type of generalization involves predicting well for new pairs of responses for users whose preferences have already been learned from the training data. We do so by evaluating the learnt model's ($\phi, \{\mathbf{w}_i\}_{i\in\mathcal{U}_{\text{seen}}}$ from Eq. (10)) classification accuracy (via Eq. (20)) on $\mathcal{D}_{\text{test}}^{\text{seen}}$ which contains the seen users' labels on prompts and response pairs that are not present in the training dataset $\mathcal{D}_{\text{train}}$.

2. **Generalizing to New Users (Unseen Accuracy):** This type of generalization involves predicting well for users whose data was not part of the training data $\mathcal{D}_{\text{train}}$, i.e., completely new users with unseen preferences. These users are termed as **unseen users** $\mathcal{U}_{\text{unseen}}$. These users come with few labelled data points, that is termed as $\mathcal{D}_{\text{fewshot}}$, and this is used to few-shot learn these users' preferences $\{\mathbf{w}_i\}_{i\in\mathcal{U}_{\text{unseen}}}$ by optimizing Eq. (11), keeping ϕ fixed. We evaluate the learnt preference weights' accuracy (via Eq. (20)) on $\mathcal{D}_{\text{test}}^{\text{unseen}}$ which contains the unseen users' labels on prompts and response pairs that are not present in the training dataset $\mathcal{D}_{\text{fewshot}}$.

3. **Performance as the number of dialogs increases for few-shot estimation:** It is natural that a new user gradually builds up feedback data on a LLM server. So far we were working in the setup where the LLM has already collected some feedback for the user based on some conversations, and see how the estimated preferences generalizes to future conversations. We consider how the performance varies as the LLM provider builds up multiple conversations with users. We do so by increasing the dataset size for $\mathcal{D}_i \in \mathcal{D}_{\text{fewshot}}$ while learning $\mathbf{w}_i$ via Eq. (11) for all $i \in \mathcal{U}_{\text{unseen}}$.

B.1 Architecture of Baselines and Training Hyperparameters

Functional Form of Reward Basis Function (LoRe): For all our experiments, we use a pre-trained reward model (Liu et al., 2024) to output embeddings of dimension $D = 4096$, which we denote as $\mathbf{e} : \mathcal{X} \times \mathcal{Y} \mapsto \mathbb{R}^D$. We keep this embedding function frozen and learn a linear transformation on top of these fixed embeddings. The reward basis function is thus defined as:

$$\mathbf{R}_\phi(x,y) = \mathbf{A}\mathbf{e}(x,y), \tag{21}$$

where $\mathbf{A} \in \mathbb{R}^{B\times D}$ is a learnable matrix, representing the linear transformation applied to the pre-trained embeddings. By keeping the embedding function fixed and only learning the linear transformation, we can effectively adapt the pre-trained reward model to user preferences from specific datasets while leveraging its rich feature representations.

Hyperparameters: $\mathbf{A}$ is learned by using Adam (Kingma, 2014) on Eq. (10) with learning rate 0.5. For few-shot adaptation, we use Adam with learning rate 0.1 on Eq. (11).

The number of basis B is selected from $\{2, 5, \ldots, 50\}$ through cross validation on a held out validation set.

Experimental Setup for baselines: We follow the exact training code, model architecture, and hyperparameters as described in the original implementations of PAL (Chen et al., 2024a) and VPL (Poddar et al., 2024), and run them on our benchmark datasets without modification. Detailed hyperparameter settings for PAL and VPL are listed in Table 4 and Table 5, respectively.

Parameter Efficiency of LoRe As shown in Table 6 and Figure 3, LoRe is significantly more lightweight in terms of total trainable parameters compared to PAL and VPL. While PAL and VPL rely on large MLP architectures and per-user latent representations, LoRe uses only a simple linear projection on frozen embeddings, combined with a small set of basis-user interactions. This design leads to a much more compact model, especially as the number of users increases, enabling scalable personalization without compromising effectiveness.

Table 4 The training hyperparameter setting of PAL (Chen et al., 2024a).

Hyperparameters	Values
B (Number of prototypes)	selected through cross validation from $\{2, 5, 10, \ldots, 50\}$
Batch size	4
Projectors	mlp-2layer-geul-dropout0
Learning rate of projectors	1e-4
Learning rate of user weights	5e-3
Weight decay of projectors	0.01
Weight decay of user weights	0.0
Dimension of preference embedding	512

Table 5 Hyperparameters for VPL (Poddar et al., 2024).

Hyperparameter	Value
Pair Encoder Architecture	2 layer MLP with LeakyReLU
Hidden Dimension	512
Latent Dimension	512
Learning rate	1.0000×10^{-4}
Learning rate scheduler	Cosine with 3% warmup steps
Batch size	32
Optimizer	AdamW (with weight decay = 0.001)

Table 6 Scaling of Total Trainable Parameters for VPL, PAL, and LoRe. Here, N is the number of seen users and B refers to the number of prototypes in PAL, and number of basis (rank) in LoRe.

Method	Architecture Details	Parameter Count
VPL	2 layer MLP on frozen embeddings	$4096 \times 512 + 512 \times 512 + 512$
PAL	2 layer MLP on frozen embeddings (B prototypes)	$4096 \times 512 + 512 \times 512 + 4096 \times B + B \times N$
LoRe	Linear Transformation on frozen embeddings (B basis)	$B \times 4096 + B \times N$

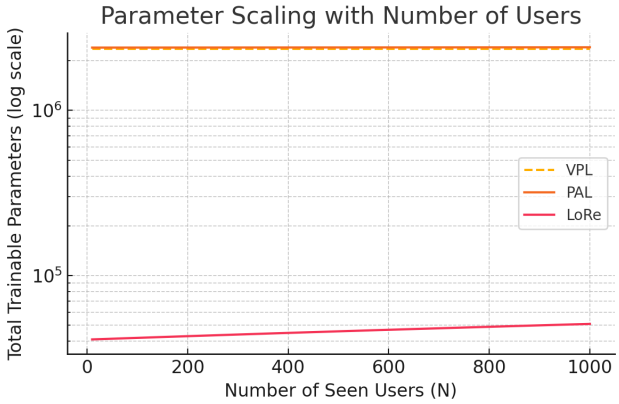

Figure 3 Trainable parameter count vs. number of seen users (log scale). LoRe scales significantly more efficiently than PAL and VPL as the number of users increases. Unlike VPL and PAL, which rely on large MLPs and high-dimensional prototype representations, LoRe uses a lightweight linear projection with shared basis vectors, resulting in dramatically fewer parameters while retaining personalization capabilities.